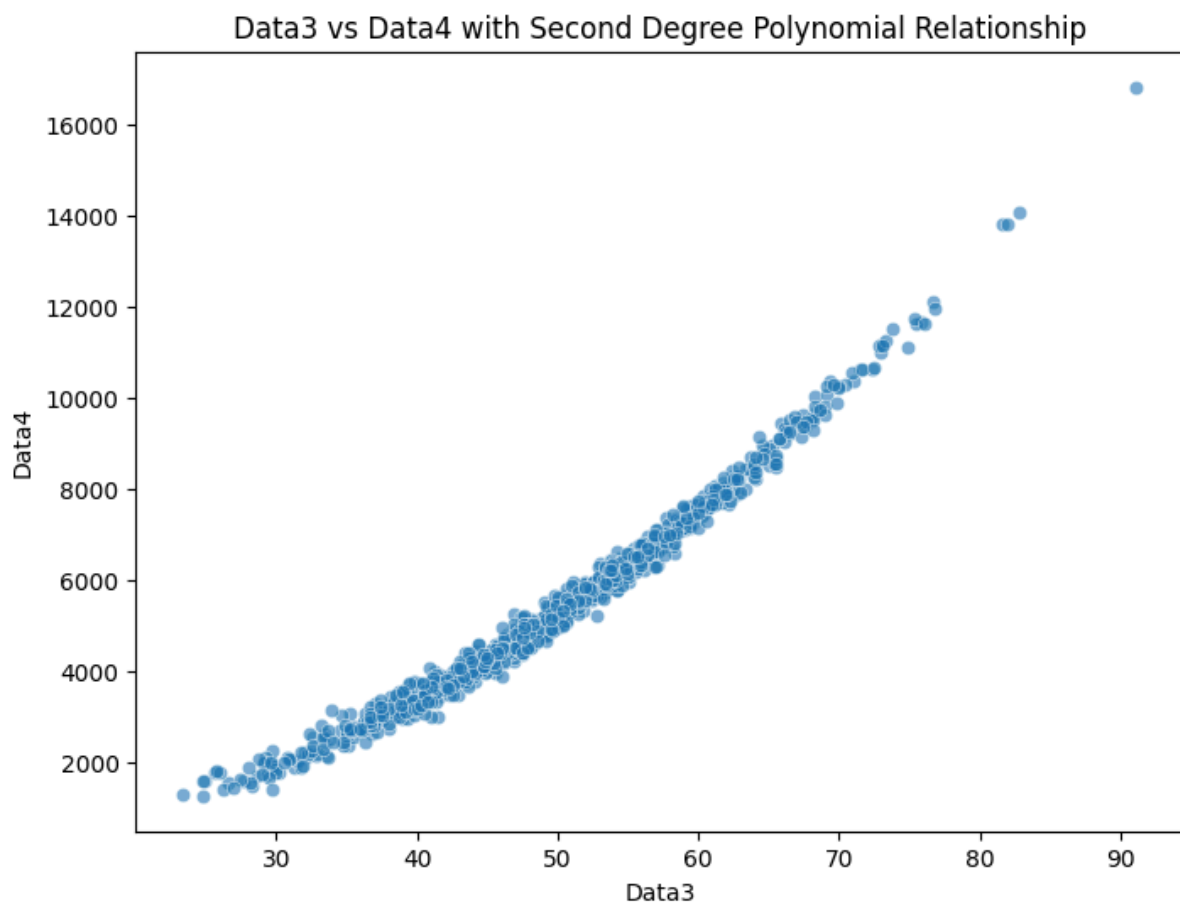


Covariance and Pearson-correlation

I created a second-degree relation between data3 and data4 to illustrate that we should be careful when interpreting Pearson's correlation. It suggests stronger linear relationship, than in the original example.

```
# prepare data3 and data4 with a polynomial relationship
data3 = 10 * randn(1000) + 50
data4 = 2 * data3**2 + 5 * data3 + 10 + (200 * randn(1000))
```



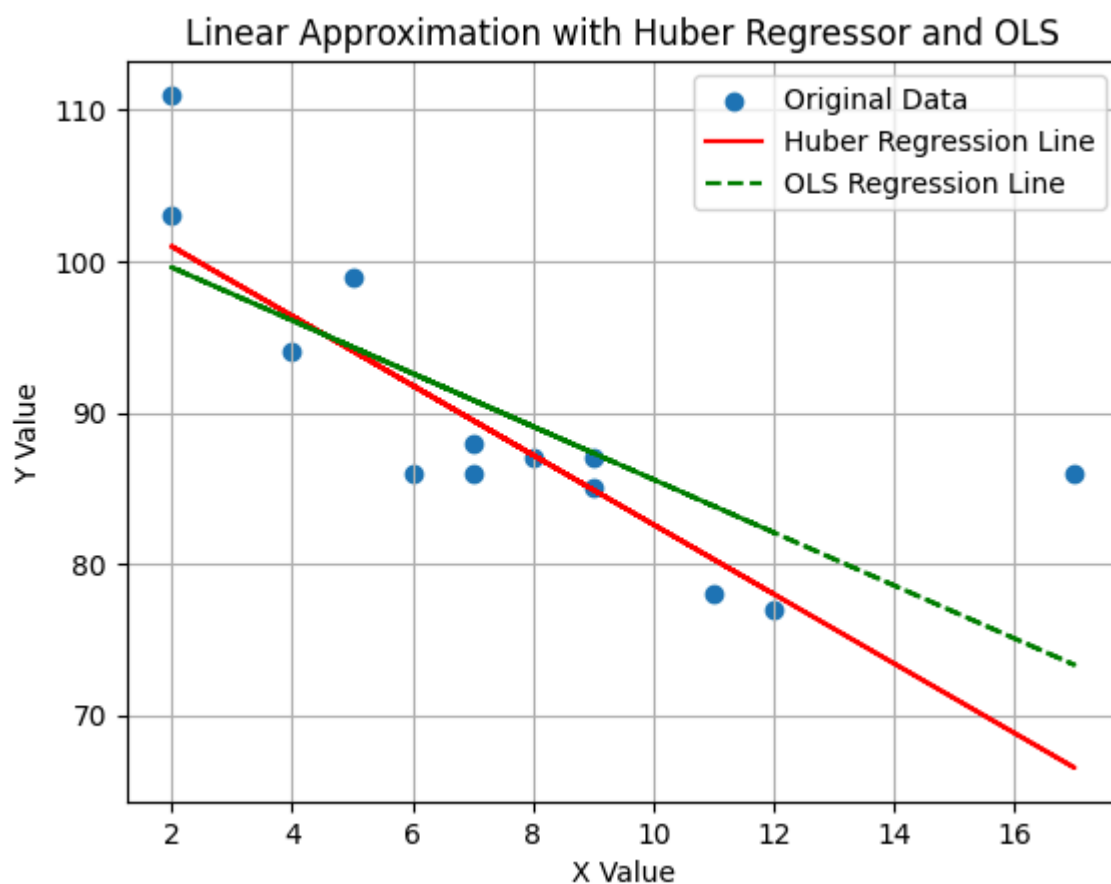
```
data3: mean=49.522 stdv=10.044
data4: mean=5361.977 stdv=2105.150
Covariance (data3, data4): 20864.446
Pearsons correlation (data3, data4): 0.986
```

Linear regression

Linear regression is not a single method but rather a broad class of models defined by a linear relationship between predictors and the target variable

In our example, we applied ordinary least squares (OLS), which estimates parameters by minimising the sum of squared residuals. While OLS is simple and widely used, it is sensitive to outliers due to the quadratic form of its loss function.

Alternative approaches, such as Huber regression, address this limitation by using a loss function that is quadratic for small residuals but linear for large ones. This makes Huber regression more robust to extreme values, providing more stable estimates in the presence of noise or outliers, while still retaining the interpretability of linear models.



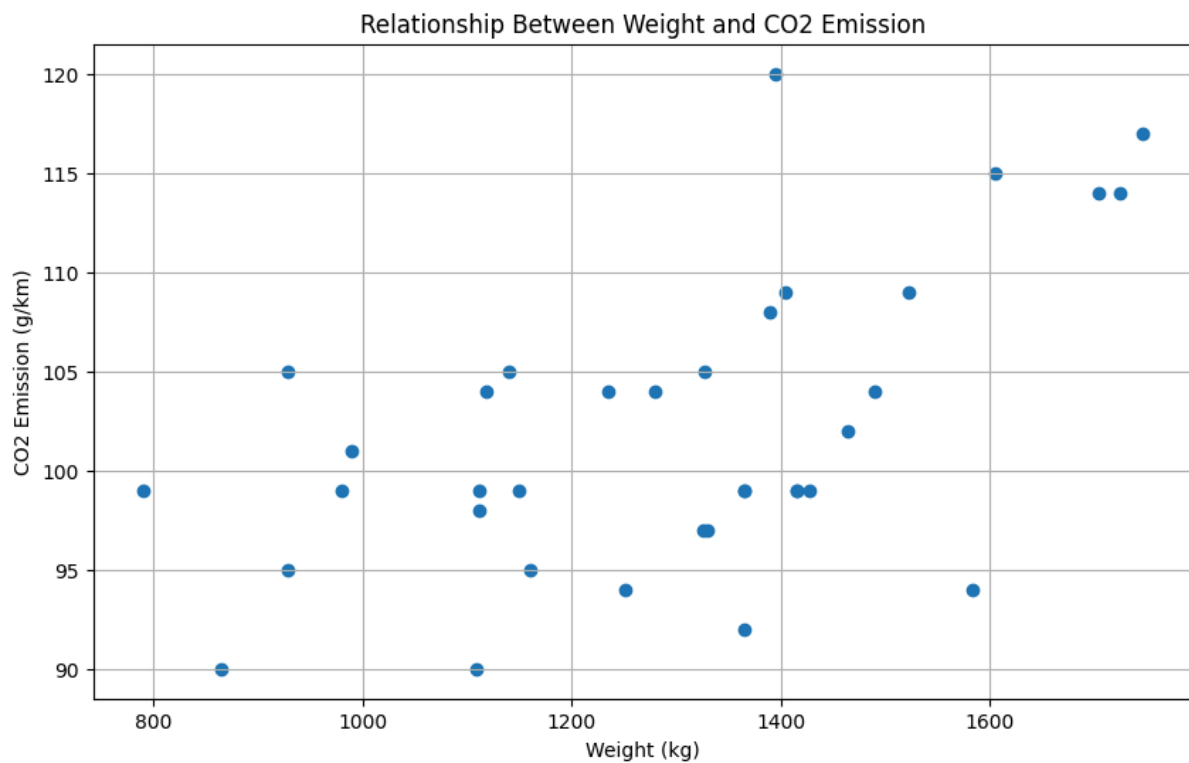
Multiple linear regression

To enhance the prediction model, another explanatory variable can be added: the Car brand. Since Car is a categorical variable, we can use one-hot encoding to convert it into a numerical format that the linear regression model can understand.

The original model can be improved by this:

Original Model R-squared: 0.3766

Extended Model R-squared: 0.5846



Polynomial regression

I changed the degree of the polynomial to several values in the original example. A higher degree comes with higher computational costs. Generally, we should find the balance between accuracy and resources, or over- vs. underfitting. In this case degree 2 is clearly not so good, while degree 4 doesn't really improve the degree 3 model.

```
1 import numpy
2 from sklearn.metrics import r2_score
3
4 x = [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]
5 y = [100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]
6
7 mymodel = numpy.poly1d(numpy.polyfit(x, y, 2))
8
9 print(r2_score(y, mymodel(x)))
```

... 0.759777160189589

```
1 import numpy
2 from sklearn.metrics import r2_score
3
4 x = [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]
5 y = [100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]
6
7 mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))
8
9 print(r2_score(y, mymodel(x)))
```

... 0.9432150416451026

```
1 import numpy
2 from sklearn.metrics import r2_score
3
4 x = [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]
5 y = [100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]
6
7 mymodel = numpy.poly1d(numpy.polyfit(x, y, 4))
8
9 print(r2_score(y, mymodel(x)))
```

... 0.9542030834699505